

Eviction-Based Memory Management for 3D Gaussian Splatting SLAM

Project Repository

<https://github.com/DevGosho160/Memory-Management-for-3D-Gaussian-Splatting-SLAM>

GitHub submission tag: `proposal-v1` (create this tag from the commit that contains this PDF).

Team and Responsibilities

- **Devon Goshorn** (DevGosho160) — systems and infrastructure: extend MonoGS with a configurable VRAM budget, an eviction queue, instrumentation, and a reproducible experiment runner.
- **Phillip Minghao Li** (plmhcode) — algorithm design and evaluation: develop pruning heuristics, define ablations, analyze the memory-accuracy trade-off, and document results.

Introduction

3D Gaussian Splatting (3DGS) represents a scene with many Gaussian primitives and supports high-quality real-time rendering. In 3DGS-based SLAM, the map grows as new observations arrive, so GPU-memory demand can increase throughout a long trajectory. This is a practical barrier for real-time robotics and mobile platforms, where memory and power are fixed.

We will extend MonoGS with an explicit memory budget and evaluate online eviction policies that keep the map within that budget while minimizing loss in localization accuracy and rendered-scene quality.

Problem Statement

Given an RGB-D or monocular RGB sequence processed by MonoGS, maintain an active Gaussian map whose GPU-memory use stays below a specified budget while preserving trajectory and rendering quality.

Our initial benchmarks will be selected **Replica** synthetic RGB-D scenes and **TUM RGB-D** sequences supported by MonoGS. Replica provides controlled ground truth for trajectories and rendering; TUM RGB-D adds real sensor data and realistic motion/noise. We will start with a small fixed subset of scenes and expand if runtime permits.

The main outcome will be a memory-quality trade-off curve. For each budget, we will report peak VRAM, active Gaussian count, mapping speed, trajectory error, and rendering quality. We expect informed eviction to preserve more useful map content than naive deletion at the same budget while avoiding out-of-memory failures.

Research Questions and Hypotheses

RQ1. Which online pruning heuristic produces the lowest trajectory error under a fixed GPU-memory budget?

RQ2. How much rendering and mapping quality is lost as the memory budget becomes smaller?

H1. A hybrid policy that applies cheap scoring continuously and expensive spatial or loop-closure-aware pruning only at selected events will achieve lower trajectory error than FIFO, random deletion, or opacity-only pruning at the same budget.

H2. Retaining recently observed, frequently visible, high-contribution Gaussians while preserving spatial coverage will provide a better memory-quality trade-off than age-only eviction.

Technical Approach

We will build on the public MonoGS implementation rather than developing a SLAM system from scratch. The modified pipeline will measure active Gaussian count and GPU memory, accept a configurable budget, trigger eviction as the map approaches that budget, and log every pruning event.

Each Gaussian will be scored from signals available during mapping: observation recency, visibility count, opacity/rendering contribution, and spatial location. We will compare:

- **No-budget MonoGS** — the ordinary, growing-map reference.
- **FIFO / age-based** — evict the least recently observed Gaussians.
- **Random** — a deliberately weak budgeted baseline.
- **Opacity/contribution-based** — remove low-opacity or low-contribution Gaussians.
- **Usage-aware hybrid** — rank candidates by normalized recency, visibility, and contribution. A spatial-coverage constraint will avoid deleting all support from a local region; costly reevaluation will run periodically or after loop-closure-related updates.

We will test budgets at 25%, 50%, and 75% of the unbounded baseline’s peak memory, plus an absolute peak-VRAM measurement. Within each comparison, input sequence, MonoGS settings, and hardware will remain fixed.

Evaluation

Quantitative evaluation

For each scene, policy, and memory budget, we will report:

- peak GPU memory and active Gaussian count;
- Absolute Trajectory Error (ATE RMSE), after standard trajectory alignment where ground truth is available;
- PSNR, SSIM, and LPIPS on evaluation viewpoints when supported by the benchmark;
- mapping throughput and eviction overhead; and
- completion rate and out-of-memory failures.

We will summarize results across scenes with mean and standard deviation. The principal comparison is the hybrid policy versus the unbounded reference and each budgeted baseline at equal memory budgets. When feasible, we will repeat runs with fixed seeds and use paired comparisons across scenes.

Qualitative evaluation

We will include memory-versus-ATE and memory-versus-rendering-quality plots, time-series plots of VRAM use and Gaussian count, trajectory overlays against ground truth, and rendered-view comparisons that reveal artifacts from aggressive eviction. Success means enforcing the budget without out-of-memory failures while achieving a better trade-off than naive budgeted baselines.

Related Work

Kerbl et al. introduced 3D Gaussian Splatting as an explicit, differentiable scene representation with high-quality real-time rendering. MonoGS, GS-SLAM, SplatAM, and Photo-SLAM then applied Gaussian maps to online dense SLAM. These systems motivate our trajectory and rendering metrics, but their map-maintenance mechanisms do not isolate the effect of a strict, user-configurable GPU-memory budget.

The closest systems work is DiskChunGS, which bounds GPU use by partitioning a scene into spatial chunks and swapping inactive chunks to disk. It demonstrates that large-scale Gaussian SLAM can avoid GPU-memory failures, but changes the map architecture and relies on out-of-core storage. Our project instead retains the MonoGS map structure and studies what can be gained from online eviction when only a bounded active map is available.

Pruning and compaction work supplies candidates for the eviction score. PUP 3D-GS uses uncertainty/sensitivity to identify dispensable primitives; SafeguardGS studies safer cross-view and pixel-wise pruning scores; MemGS merges geometrically similar Gaussians to reduce memory use. These papers largely focus on static reconstruction, rendering, or a different SLAM pipeline. We will adapt inexpensive signals that can be computed online in MonoGS, then evaluate them under matched VRAM budgets with localization metrics. The novelty claim is deliberately limited to this systems evaluation and budget controller, not to inventing Gaussian pruning itself.

Expected Deliverables

- A reproducible MonoGS-based implementation with VRAM budgeting and pluggable eviction policies;
- configuration files and scripts for benchmark runs;
- logs, plots, and qualitative visualizations;

- a final report with design decisions, ablations, limitations, and results; and
- a tagged GitHub release containing the submission snapshot.

Timeline and Milestones

Period	Milestone
Weeks 1–2	Set up MonoGS, reproduce a baseline, and add memory instrumentation.
Weeks 3–4	Add the budget controller and FIFO/random/opacity baselines.
Weeks 5–6	Implement the usage-aware hybrid policy and run ablations.
Weeks 7–8	Run Replica and TUM RGB-D experiments; collect plots and visual outputs.
Final period	Analyze results, document limitations, polish reproducibility materials, and tag the final submission.

Risks and Mitigations

- **MonoGS setup or dataset runtime is costly.** Begin with a small fixed scene subset and publish exact configurations.
- **A strict budget causes instability.** Sweep budgets gradually and retain unbounded and simple policy baselines.
- **A sequence lacks a supported metric.** Use its available benchmark metrics and report exclusions explicitly.
- **Eviction overhead offsets memory gains.** Log policy runtime separately and favor sparse expensive updates.

Reproducibility Plan

The repository will record the MonoGS version, environment specification, GPU/CUDA details, dataset acquisition instructions, scene lists, random seeds, and hyperparameters. Versioned configurations and scripts will produce machine-readable logs and figures. Large datasets and outputs will not be committed; their source URLs and expected directory layout will be documented.

References

1. Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM TOG*, 2023.
2. Hidenobu Matsuki, Riku Murai, Paul H. J. Kelly, and Andrew J. Davison. Gaussian Splatting SLAM. *CVPR*, 2024. (MonoGS)
3. C. Yan et al. GS-SLAM: Dense Visual SLAM with 3D Gaussian Splatting. 2023.
4. C. Feldmann et al. DiskChunGS: Large-Scale 3D Gaussian SLAM Through Chunk-Based Memory Management. 2025.
5. A. Hanson et al. PUP 3D-GS: Principled Uncertainty Pruning for 3D Gaussian Splatting. 2024.
6. Y. Lee, Z. Zhang, and D. Fan. SafeguardGS: 3D Gaussian Primitive Pruning While Avoiding Catastrophic Scene Destruction. 2024.
7. B. Yin-long et al. MemGS: Memory-Efficient Gaussian Splatting for Real-Time SLAM. 2025.
8. S. Keetha et al. SplatTAM: Splat Track & Map 3D Gaussians for Dense RGB-D SLAM. *CVPR*, 2024.
9. Huajian Huang et al. Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping. *CVPR*, 2024.
10. Jürgen Sturm et al. A Benchmark for the Evaluation of RGB-D SLAM Systems. *IROS*, 2012.
11. Julian Straub et al. The Replica Dataset: A Digital Replica of Indoor Spaces. arXiv, 2019.